KYAMBOGO UNIVERSITY

FACULTY OF ENGINEERING · DEPARTMENT OF ELECTRICAL AND ELECTRONICS

TETE 3102: MICROCONTROLLERS AND APPLICATIONS

LABORATORY REPORT: TASK 1

Theoretical Exercises, Addressing Math & Protocol Mechanics

Mukama Robert (26/U/ETE/21285/PE)
Nabudere Zerubabel (25/U/ETE/05465/PE)
Nanfuka Shakirah (25/U/ETE/05468/PE)
Kidega Wilkie Cephas (25/U/ETE/05460/PE)

Group Number: Group 4
Platforms: STM32F411RE & Arduino Uno
Course Supervisor: Dr. Mugerwa Dickson PhD

OBJECTIVE & LAB FOCUS

The objective of this task is to master the foundational mechanics of the Inter-Integrated Circuit (I^2C) protocol, including 7-bit to 8-bit wire addressing transformations, physical bus timing constraints, pull-up resistor sizing under capacitive loading, open-drain bus arbitration, and microcontroller driver architectures (Polling vs. Interrupt vs. DMA).

Exercise 1: Addressing Conversions (7-bit vs. 8-bit Wire Frames)

THEORETICAL FOUNDATION & PROTOCOL RULES

In the I^2C specification (NXP UM10204), a standard slave address is 7 bits long ($A_6 \ldots A_0$). When transferred across the physical serial bus, the address occupies the 7 most significant bits of the first byte transmitted after a START condition. Bit 0 ($R/\overline{W}$) defines the direction:

$$\text{Address Byte (Wire)} = (\text{7-bit Address} \ll 1) \mid (R/\overline{W}) \tag{1}$$

where $R/\overline{W} = 0$ designates a **Master Write** operation and $R/\overline{W} = 1$ designates a **Master Read** operation.

Step-by-Step Conversions

(a) Given 7-bit Address: 0x50

- Binary representation (7-bit): $0x50 = 1010000_2$
- Bit-shift left by 1 bit: $1010000_2 \ll 1 = 10100000_2$
- **Write Address Byte** ($R/\overline{W} = 0$): $10100000_2 =$ **0xA0**
- **Read Address Byte** ($R/\overline{W} = 1$): $10100001_2 =$ **0xA1**
- *Common Device Context:* Standard 24C32/24C64 I2C EEPROM.

(b) Given 7-bit Address: 0x3F

- Binary representation (7-bit): $0x3F = 0111111_2$
- Bit-shift left by 1 bit: $0111111_2 \ll 1 = 01111110_2$
- **Write Address Byte** ($R/\overline{W} = 0$): $01111110_2 =$ **0x7E**
- **Read Address Byte** ($R/\overline{W} = 1$): $01111111_2 =$ **0x7F**

- *Common Device Context:* PCF8574A I2C LCD backpack (default address with jumpers open).

(c) Given Address Byte: 0xD1

- Binary on wire: $0xD1 = 11010001_2$
- Bit 0 is 1 $\implies$ **Read Byte**.
- Extract 7-bit address by shifting right by 1: $11010001_2 \gg 1 = 01101000_2 =$ **0x68**.
- Corresponding **Write Address Byte** ($R/\overline{W} = 0$): $11010000_2 =$ **0xD0**.
- *Common Device Context:* DS3231 High-Precision Real-Time Clock (RTC).

(d) Given Address Byte: 0x78 (Designated Write)

- Binary on wire: $0x78 = 01111000_2$
- Bit 0 is 0 $\implies$ **Write Byte**.
- Extract 7-bit address: $01111000_2 \gg 1 = 00111100_2 =$ **0x3C**.
- Corresponding **Read Address Byte** ($R/\overline{W} = 1$): $01111001_2 =$ **0x79**.
- *Common Device Context:* SSD1306 0.96-inch 128×64 OLED display.

(e) Given 7-bit Binary Address: 0110000_2

- Hexadecimal 7-bit equivalent: $0110000_2 =$ **0x30**
- Bit-shift left by 1 bit: $0110000_2 \ll 1 = 01100000_2$
- **Write Address Byte** ($R/\overline{W} = 0$): $01100000_2 =$ **0x60**
- **Read Address Byte** ($R/\overline{W} = 1$): $01100001_2 =$ **0x61**

Table 1: Consolidated Summary of Address Conversions

Item	7-bit Hex	7-bit Binary	Write Byte (Hex)	Read Byte (Hex)	Typical Device Identity
(a)	`0x50`	1010000_2	`0xA0`	`0xA1`	24C32 / AT24C EEPROM
(b)	`0x3F`	0111111_2	`0x7E`	`0x7F`	PCF8574A LCD Backpack
(c)	`0x68`	1101000_2	`0xD0`	`0xD1`	DS3231 RTC Module
(d)	`0x3C`	0011100_2	`0x78`	`0x79`	SSD1306 Graphic OLED
(e)	`0x30`	0110000_2	`0x60`	`0x61`	Lecture Slide Example

Exercise 2: Complete Bus Frame Timing Diagram

Scenario: Master writes the byte `0x55` (01010101_2) to a slave device at 7-bit address `0x48` (1001000_2).

Mathematical & Protocol Construction

- **7-bit Slave Address:** `0x48` ($A_6 = 1, A_5 = 0, A_4 = 0, A_3 = 1, A_2 = 0, A_1 = 0, A_0 = 0$).
- **Direction Bit ($R/\overline{W}$):** 0 (Write).
- **Wire Address Byte:** $(0x48 \ll 1) \mid 0 = 10010000_2 =$ **0x90**.
- **Data Byte:** $0x55 = 01010101_2$.
- **Acknowledge Responses:** Driven **LOW** by the slave during the 9th clock of the address phase and 9th clock of the data phase.

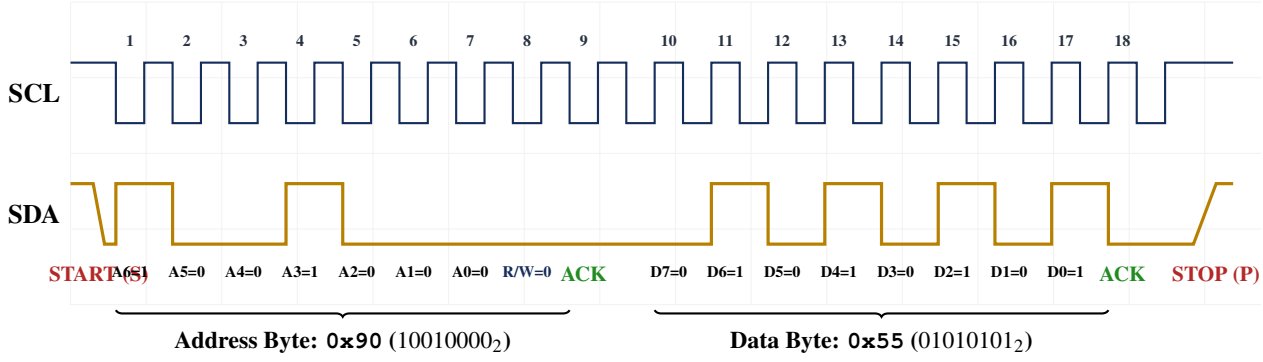


Figure 1: Complete physical bus transaction waveform for writing `0x55` to address `0x48`.

Exercise 3: Pull-Up Resistor Calculation ($R_{p(\min)}$ and $R_{p(\max)}$)

Operating Parameters Given

- Bus Supply Voltage: $V_{DD} = 5.0$ V
- Total Parasitic Bus Capacitance: $C_b = 150$ pF
- Standard-mode (100 kHz): maximum permissible rise time $t_{r(\max)} = 1000$ ns
- Fast-mode (400 kHz): maximum permissible rise time $t_{r(\max)} = 300$ ns
- Device sink current: $I_{OL} = 3$ mA at $V_{OL(\max)} = 0.4$ V

Mathematical Derivation of Limits

1. Minimum Pull-Up Resistor ($R_{p(\min)}$): Determined by the maximum sink capability of open-drain output FETs to guarantee $V_{OL} \leq 0.4$ V:

$$R_{p(\min)} = \frac{V_{DD} - V_{OL(\max)}}{I_{OL}} = \frac{5.0\text{ V} - 0.4\text{ V}}{3.0 \times 10^{-3}\text{ A}} = \frac{4.6\text{ V}}{0.003\text{ A}} = \mathbf{1533.33}\blacksquare \approx \mathbf{1.53}\text{ k}\blacksquare \tag{2}$$

2. Maximum Pull-Up Resistor ($R_{p(\max)}$): Governed by the RC exponential charging curve where rise time t_r is measured between 30% and 70% of V_{DD}:

$$V(t) = V_{DD}\left(1 - e^{-t/RC}\right) \implies t_r = \ln\left(\frac{1-0.3}{1-0.7}\right) R_p C_b = \ln\left(\frac{0.7}{0.3}\right) R_p C_b \approx 0.8473 \cdot R_p C_b \tag{3}$$

$$R_{p(\max)} = \frac{t_{r(\max)}}{0.8473 \cdot C_b} \tag{4}$$

- **Case (a): Standard-mode** (100 **kHz,** $t_r \leq 1000$ **ns):**

$$R_{p(\max,100k)} = \frac{1000 \times 10^{-9}\ \text{s}}{0.8473 \times 150 \times 10^{-12}\ \text{F}} = \frac{1000}{0.127095}\ \Omega = \mathbf{7868.1}\blacksquare \approx \mathbf{7.87}\ \text{k}\blacksquare \tag{5}$$

  Allowable Range: 1.53 kΩ $\leq R_p \leq$ 7.87 kΩ.
  Chosen Standard E24 Resistor: 4.7 k■ (typical laboratory default).

- **Case (b): Fast-mode** (400 **kHz,** $t_r \leq 300$ **ns):**

$$R_{p(\max,400k)} = \frac{300 \times 10^{-9}\ \text{s}}{0.8473 \times 150 \times 10^{-12}\ \text{F}} = \frac{300}{0.127095}\ \Omega = \mathbf{2360.4}\blacksquare \approx \mathbf{2.36}\ \text{k}\blacksquare \tag{6}$$

  Allowable Range: 1.53 kΩ $\leq R_p \leq$ 2.36 kΩ.
  Chosen Standard E24 Resistor: 2.2 k■ (or 1.8 kΩ).

HARDWARE & PROTOCOL TROUBLESHOOTING NOTE

Notice that a 4.7 kΩ pull-up resistor violates the Fast-mode specification when $C_b = 150$ pF, yielding $t_r = 0.8473 \times 4700 \times 150$ pF $= 597$ ns, which is double the permissible 300 ns limit. This causes high-speed edge rounding and communication failures.

Exercise 4: Sensor Register-Read Transaction Duration

Transaction Sequence:

$$\text{START} \rightarrow \underbrace{\text{addr+W}}_{9\text{ clocks}} \rightarrow \underbrace{\text{register}}_{9\text{ clocks}} \rightarrow \text{Sr} \rightarrow \underbrace{\text{addr+R}}_{9\text{ clocks}} \rightarrow \underbrace{\text{Data 1}}_{9\text{ clocks}} \rightarrow \cdots \rightarrow \underbrace{\text{Data 6}}_{9\text{ clocks}} \rightarrow \text{STOP}$$

Clock Cycle Accounting

Every transmitted or received byte consists of 8 data bits followed by 1 ACK/NACK bit (9 SCL clock pulses per byte).

- Total bytes moved = 1 (addr+W) + 1 (register) + 1 (addr+R) + 6 (data bytes) = **9 bytes total**.
- Total clock periods (N_{clocks}) = 9 bytes × 9 clocks/byte = **81** SCL clock cycles.

Time Calculations (Ignoring framing overhead)

(a) At 100 kHz Standard-mode ($T_{\text{SCL}} = 10.0\,\mu$s):

$$t_{\text{transfer, 100k}} = 81 \times 10.0\,\mu\text{s} = \mathbf{810.0}\,\mu\text{s} \quad (0.81\text{ ms}) \tag{7}$$

(b) At 400 kHz Fast-mode ($T_{\text{SCL}} = 2.5\,\mu$s):

$$t_{\text{transfer, 400k}} = 81 \times 2.5\,\mu\text{s} = \mathbf{202.5}\,\mu\text{s} \quad (\approx 0.20\text{ ms}) \tag{8}$$

Speedup factor: $810/202.5 = \mathbf{4.0}\times$ throughput increase.

Exercise 5: Rationale for Master NACK on Final Read Byte

THEORETICAL FOUNDATION & PROTOCOL RULES

During a Master-Receive transaction, the slave drives the SDA line to present successive data bits while the master drives the SCL clock. On the 9th clock pulse of each byte, the roles reverse: the master drives SDA to acknowledge the byte.

Why Master sends NACK before STOP:

1. If the master were to send an **ACK** (holding SDA LOW) on the final byte, the slave would prepare to transmit the subsequent byte, immediately asserting its next data bit onto the SDA line on the next falling edge of SCL.
2. If that subsequent bit happens to be a logic 0, the slave pulls SDA LOW.
3. To generate a valid **STOP (P)** condition, the master must cause a low-to-high transition on SDA while SCL is held HIGH.
4. However, because the bus is a wired-AND configuration, if the slave is pulling SDA LOW, the master **cannot release SDA HIGH**. The master is locked out from generating a valid STOP condition, hanging the bus.
5. Transmitting a **NACK** (master releases SDA HIGH on the 9th clock) explicitly informs the slave that transmission has ended. The slave releases SDA into high-impedance mode, allowing the master to cleanly toggle SDA from LOW to HIGH during SCL HIGH to issue the STOP condition.

Exercise 6: Address Conflicts on a Shared Bus

The Problem: I^2C uses in-band addressing broadcast across shared SDA/SCL lines. If two slave devices share identical 7-bit addresses (e.g., two fixed-address sensors at `0x68`):

- Both slaves detect their address and simultaneously pull SDA low to assert ACK.
- During read transactions, both slaves simultaneously drive SDA according to their internal registers. If Slave 1 attempts to output a 1 and Slave 2 outputs a 0, the wired-AND configuration forces the line to 0. The master receives corrupted data, and bus arbitration mechanisms fail.

Two Engineering Solutions:

1. **Hardware Pin Strapping (A_0, A_1, A_2 Jumpers):** Most modern I^2C chips (such as the PCF8574 expander or 24C EEPROMs) include physical address pins. Tying these pins to V_{DD} or GND offsets the lowest address bits, assigning unique 7-bit addresses (e.g., `0x20` through `0x27`).
2. **I2C Multiplexer / Bus Switch IC (e.g., TCA9548A):** When devices have strictly fixed, unchangeable addresses, an active I^2C multiplexer is placed between the master and slaves. The master commands the multiplexer over address `0x70` to dynamically connect only one sub-channel at a time.

Exercise 7: Open-Drain Physics vs. Push-Pull Hazards

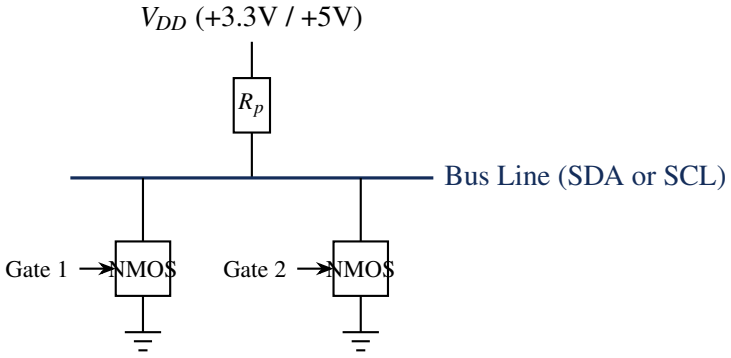


Figure 2: Wired-AND Open-Drain architecture of the I^2C bus.

Why Open-Drain is Mandatory

1. **Wired-AND Logic:** A line is logic HIGH only when **all** connected open-drain transistors are turned OFF (allowing R_p to pull the line up). If any single device turns its transistor ON, the line is pulled LOW.
2. **Non-Destructive Clock Stretching & Multi-Master Arbitration:** A slave can safely hold SCL LOW while servicing an interrupt without causing damage. In multi-master systems, two masters transmitting different bits do not create bus contention.
3. **Safe Voltage Level Interoperability:** Devices operating at 3.3 V and 5 V can coexist on the same physical line if the pull-up resistor is tied to 3.3 V and the 5 V devices tolerate 3.3 V V_{IH} logic thresholds.

What Happens if Push-Pull Outputs are Used?

In a push-pull driver stage, active PMOS and NMOS transistors actively drive the pin to V_{DD} and GND respectively.

- If Device 1 drives the line HIGH (V_{DD}) while Device 2 drives the line LOW (GND), a direct low-impedance short circuit is formed from V_{DD} through the PMOS of Device 1 and the NMOS of Device 2 to GND.
- The resulting shoot-through fault current:

$$I_{\text{short}} = \frac{V_{DD}}{R_{\text{ON, PMOS}} + R_{\text{ON, NMOS}}} \approx \frac{5\text{ V}}{20\,\Omega + 20\,\Omega} = 125\text{ mA} \tag{9}$$

  This current far exceeds the maximum absolute rating of MCU I/O pins (typically 25 mA on STM32/ATmega), permanently destroying the output pads through thermal failure.

Exercise 8: STM32 HAL Driver Styles: Polling, Interrupt & DMA

Table 2: Comprehensive Comparison of STM32 HAL I2C Operational Modes

Metric	Polling (Blocking)	Interrupt (IT) Mode	Direct Memory Access (DMA)
HAL API Calls	`HAL_I2C_Master_Transmit` `HAL_I2C_Mem_Read`	`HAL_I2C_Master_Transmit_IT` `HAL_I2C_Mem_Read_IT`	`HAL_I2C_Master_Transmit_DMA` `HAL_I2C_Mem_Read_DMA`
CPU Utilization	$\approx$ 100% during transfer (CPU trapped in delay loop).	Very Low; CPU services short ISR per byte transfer.	Near 0%; hardware controller streams directly to RAM.
Return Timing	Blocks until transfer is complete or timeout expires.	Returns execution to caller immediately.	Returns execution to caller immediately.
Completion Signal	Return code (`HAL_OK`, `HAL_TIMEOUT`).	ISR Callback (`HAL_I2C_MemRxCpltCallback`).	DMA completion interrupt callback.
Buffer Safety	Local stack variables are safe.	Must use global/static buffers (stack will corrupt).	Must use persistent memory (DMA writes asynchronously).
When to Choose	**Short, infrequent transfers**: bus scanner, single configuration writes, sensor initialization.	**Low-volume background tasks**: reading RTC every 1 sec without freezing UI or motor loops.	**Large data payloads**: streaming 1024-byte framebuffers to SSD1306 OLED, high-speed ADC logging.